Task-Dependent Length Generalization in Transformers: An Empirical Study

Hridish Goswami

Abstract

This paper presents a two-part investigation into deep learning built from first principles. Part A implements a fully connected neural network without automatic differentiation, applied to MNIST digit classification, and documents a specific and mechanistically understood failure mode, a form of activation collapse arising from the interaction of ReLU, weight initialization, and learning rate, together with an ablation study of optimization strategies. Part B extends this first-principles approach to a Transformer encoder implemented from scratch, and uses it to investigate an open question in the literature: whether a model's positional encoding scheme affects its ability to generalize to sequence lengths beyond those seen during training. Two synthetic tasks differing in their dependence on token position, sequence reversal and element counting, are used to test three positional encoding schemes: sinusoidal, learned, and none. Results show that both the necessity of positional encoding and the relative performance of different schemes are task-dependent: no positional encoding fails outright on the strongly positional reversal task but partially succeeds on the weakly positional counting task, while sinusoidal and learned encodings perform indistinguishably on reversal but diverge substantially on counting. No configuration tested achieves robust generalization to substantially longer sequences, consistent with prior findings on length generalization in Transformers. These results suggest that single-task evaluations of positional encoding schemes may understate how sensitive such comparisons are to task structure.

1. Introduction

Modern deep learning is largely mediated by high-level frameworks: a single call to an optimizer's .step() method hides the chain rule, and a pre-built attention layer hides the mechanics of self-attention. This abstraction is productive for building systems quickly, but it can leave a gap between using an architecture and understanding it. That gap becomes apparent precisely when something fails to train and the cause is not obvious from the framework's error messages. This paper addresses that gap directly, by implementing two architectures from first principles and using each implementation as the basis for a concrete empirical investigation.

Part A implements a fully connected neural network without automatic differentiation, training it on MNIST digit classification. During development, an initial implementation failed to train at all, collapsing to a single predicted class. Diagnosing this failure, and understanding exactly why it occurred, is a compact and instructive case study in how activation functions, weight initialization, and learning rate interact. It also motivates the ablation study presented in Section 2.

Part B extends this approach to the Transformer architecture [1], implemented with automatic differentiation, and uses it to investigate an open question: does a Transformer's choice of positional encoding scheme affect its ability to generalize to sequence lengths beyond those seen during training? This question, known as length generalization, remains genuinely unsettled in the literature. Press et al. [2] introduced ALiBi, a relative encoding scheme, and showed it substantially outperforms sinusoidal and rotary encodings on extrapolation to longer sequences. A more recent, larger-scale study by Kazemnejad et al. [3] found instead that omitting positional encoding entirely (NoPE) outperformed ALiBi, rotary, and learned absolute encodings on a suite of reasoning and arithmetic tasks, suggesting that explicit positional information may not be necessary, or may even be counterproductive, for length generalization in some settings.

This work does not attempt to resolve this disagreement at scale. Instead, it investigates a narrower question that helps clarify it: does the task itself determine whether positional encoding is necessary at all, and is the relative ranking of encoding schemes stable across tasks that differ in how strongly they depend on position? Two synthetic tasks are constructed for this purpose and described in Section 3.4.

Section 2 presents the from-scratch neural network, its failure mode, and an accompanying ablation study. Section 3 presents the Transformer implementation and the two length-generalization experiments. Section 4 reviews related work. Section 5 discusses both parts jointly and states limitations. Section 6 concludes.

2. Part A: A Neural Network from First Principles

2.1 Architecture and Implementation

A fully connected feedforward network was implemented using only elementary NumPy array operations, without an automatic differentiation library, to classify handwritten digits from MNIST. The architecture consists of 784 input units (flattened, normalized 28x28 pixel values), two hidden layers of 32 units with ReLU activation, and a 10-unit softmax output layer. Weights were initialized following He et al. [4], which scales initial values by sqrt(2/n_in) rather than a fixed constant. The importance of this choice is demonstrated directly in Section 2.2.

2.2 A Failure Mode: Activation Collapse

An initial implementation used a naive initialization scheme, with weights scaled by a fixed constant of 0.01, together with a learning rate of 0.1. This implementation failed to train: training accuracy remained fixed at 11.25% throughout, indicating that the network had collapsed to predicting a single class for every input.

The underlying mechanism is straightforward once identified. Early in training, the combination of naive initialization and an aggressive learning rate drove a large fraction of hidden-unit pre-activations into strongly negative values. Since the derivative of ReLU is exactly zero for negative inputs, these units ceased to receive any gradient signal at all. Once a unit enters this state it cannot recover under further training, because a zero gradient produces no weight update regardless of the magnitude of the error elsewhere in the network. This phenomenon is commonly referred to as "dying ReLU." As a result, the network's effective capacity was substantially reduced before training had meaningfully progressed.

This was resolved by two joint changes: adopting He initialization [4] and reducing the learning rate to 0.01. With both changes, training proceeded stably, reaching 78% test accuracy at the original architecture size (16-16-10) and 90.4% after widening both hidden layers to 32 units. This progression illustrates that initialization scheme, activation function, and learning rate should not be treated as independently tunable hyperparameters. A learning rate that is reasonable in isolation can produce catastrophic failure when paired with an initialization scheme that does not account for the activation function's effect on gradient propagation.

Illustrative example.

The mechanism above is made concrete with a minimal case. Consider three inputs x = [0.2, 0.9, 0.1] feeding into two output units with weights w1 = [0.5, -0.2, 0.1] and w2 = [0.3, 0.4, -0.6], giving pre-activations:

$$z1 = (0.5)(0.2) + (-0.2)(0.9) + (0.1)(0.1) = -0.07 \qquad z2 = (0.3)(0.2) + (0.4)(0.9) + (-0.6)(0.1) = 0.36$$

Suppose the resulting softmax output is a = [0.43, 0.57] against a true label y = [0, 1]. Because softmax is paired with cross-entropy loss, the output-layer error takes the simple closed form delta = a - y = [0.43, -0.43], and the gradient with respect to w1 follows directly:

$$grad(w1) = delta\_1 \,.\, x = 0.43 \times [0.2, 0.9, 0.1] = [0.086, 0.387, 0.043]$$

Propagating this error to a preceding hidden layer requires passing it backward through the weight matrix (delta_hidden = W_output^T . delta) and then multiplying element-wise by the derivative of that layer's ReLU activation. This derivative is exactly zero for any unit whose pre-activation was negative during the forward pass. This is the precise mechanism by which a unit driven negative during training is permanently excluded from further learning, irrespective of how large or informative delta is by the time it arrives.

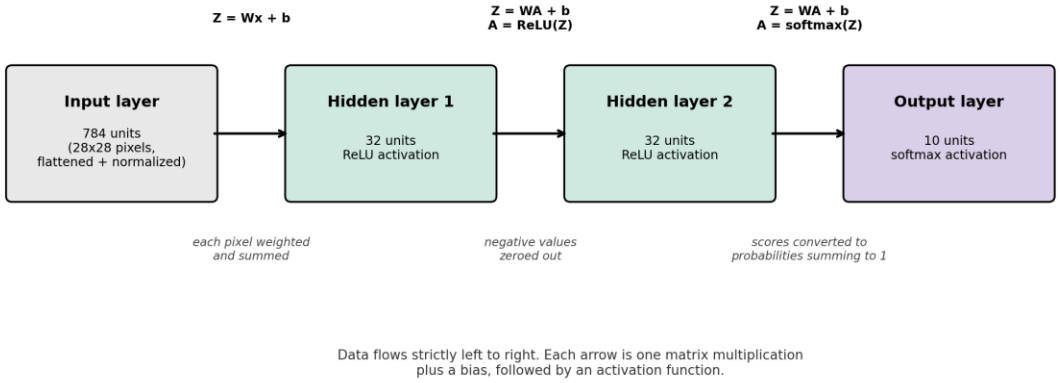

Figure 1. Forward-pass architecture of the Part A network.

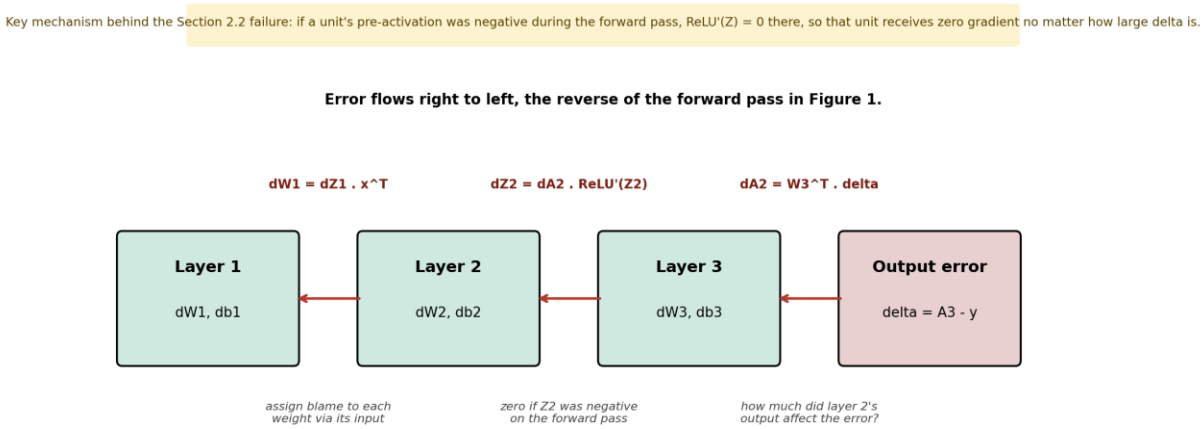

Figure 2. Backward error propagation through the Part A network.

2.3 Ablation Study: Optimization Strategy

Table I reports results from an ablation study comparing optimization strategies with architecture (32-32-10) and initialization held fixed.

Experiment	Configuration	Train acc.	Test acc.
Baseline	16-16-10, naive init, lr=0.1	0.11	0.11
Fix	16-16-10, He init, lr=0.01	0.81	0.78
Wider layers	32-32-10, He init, lr=0.01	0.90	0.90
Mini-batch	32-32-10, batch=128, 50 epochs	0.9637	0.9552
+ Momentum	same + beta=0.9	0.9634	0.9559

Table I. Ablation study results, Part A.

Switching from full-batch to mini-batch gradient descent (batch size 128) produced a substantial improvement, reaching 96.4% training and 95.5% test accuracy within 50 epochs, compared to 90%/90% after 1,500 full-batch epochs. This follows directly from the difference in update frequency: mini-batch training performs approximately 437 parameter updates per epoch, compared to one under the full-batch scheme, giving the network far more opportunities to correct itself per unit of data observed. Adding momentum (beta = 0.9) on top of mini-batch training produced no meaningful further improvement (96.34% versus 96.37% training accuracy). This is consistent with momentum's principal benefit being the mitigation of gradient noise and traversal of ill-conditioned loss curvature, neither of which is a substantial obstacle for a shallow three-layer network on a comparatively well-behaved loss surface at this batch size.

The resulting test accuracy of 95.6% is respectable relative to comparable fully connected architectures. Larger networks of this general form have reached approximately 98.8 to 98.9% on MNIST, though this remains short of results reported for architectures specifically designed to exploit image structure, such as the capsule network of Sabour et al. [5], which achieved 99.75% on this benchmark. This gap is attributable primarily to architectural choice rather than optimization: a fully connected network discards the two-dimensional spatial structure of image data upon flattening, whereas convolutional and capsule-based architectures are explicitly designed to exploit local spatial correlation.

3. Part B: Length Generalization in a From-Scratch Transformer

3.1 Motivation and Research Question

The Transformer architecture [1] has become the dominant approach for sequence modeling, in large part due to self-attention's ability to relate any two positions in a sequence directly, regardless of their distance. This is a property that recurrent architectures lack. That property does not, however, guarantee that a trained Transformer will behave sensibly on sequences longer than those it was trained on. A substantial body of recent work documents the opposite: models frequently degrade sharply once evaluated beyond their training length distribution, a phenomenon referred to as the length generalization problem [2], [3].

Because a Transformer has no inherent notion of position (self-attention itself is permutation-invariant), position must be supplied externally, via a positional encoding scheme. It is natural to ask whether the specific scheme chosen affects how well the resulting model generalizes to unseen lengths. The literature's answer to this question is presently inconsistent. Press et al. [2] introduced ALiBi, a relative encoding scheme, and showed it substantially outperforms sinusoidal and rotary encodings on extrapolation to longer sequences. Kazemnejad et al. [3] found instead that omitting positional encoding altogether (NoPE) outperformed ALiBi, rotary, and learned absolute encodings on a suite of reasoning and arithmetic tasks. This suggests that explicit positional

information may not be necessary, or may even be counterproductive, for length generalization in some settings.

This work does not attempt to resolve this disagreement at scale. Instead, it investigates a narrower question that helps clarify it: does the task itself determine whether positional encoding is necessary at all, and is the relative ranking of encoding schemes stable across tasks that differ in how strongly they depend on position? Two synthetic tasks are constructed for this purpose and described in Section 3.4.

3.2 Architecture

A Transformer encoder was implemented from scratch. PyTorch's automatic differentiation was used for gradient computation, but every architectural component (embeddings, attention, feedforward sublayers) was implemented directly rather than drawn from a pre-built Transformer module. The architecture consists of a token embedding layer, a positional encoding component that was varied across conditions (described in Section 3.3), two stacked Transformer blocks, and a final linear output layer. Each Transformer block follows the standard structure: multi-head self-attention (4 heads, model dimension 32), followed by a residual connection and layer normalization, followed by a position-wise feedforward sublayer (hidden dimension 128) with a second residual connection and layer normalization.

The residual connections and layer normalization warrant brief comment, as they address a concern directly related to Section 2. Without a residual pathway, a deep stack of transformations must, in effect, relearn how to preserve useful information at every layer. The residual connection instead allows each sublayer to learn only a correction to its input, an idea first introduced for very deep convolutional networks by He et al. [6], which substantially eases optimization. Layer normalization serves a related purpose to the careful weight initialization discussed in Section 2.1: it keeps the scale of activations consistent as they pass through successive layers, preventing the kind of instability that motivated the switch to He initialization in Part A.

Architecture and optimizer settings (model dimension, number of heads, feedforward dimension, number of layers, learning rate) were held fixed across every experimental condition, so that observed differences in outcome can be attributed to the positional encoding scheme and task design rather than to confounding architectural variation.

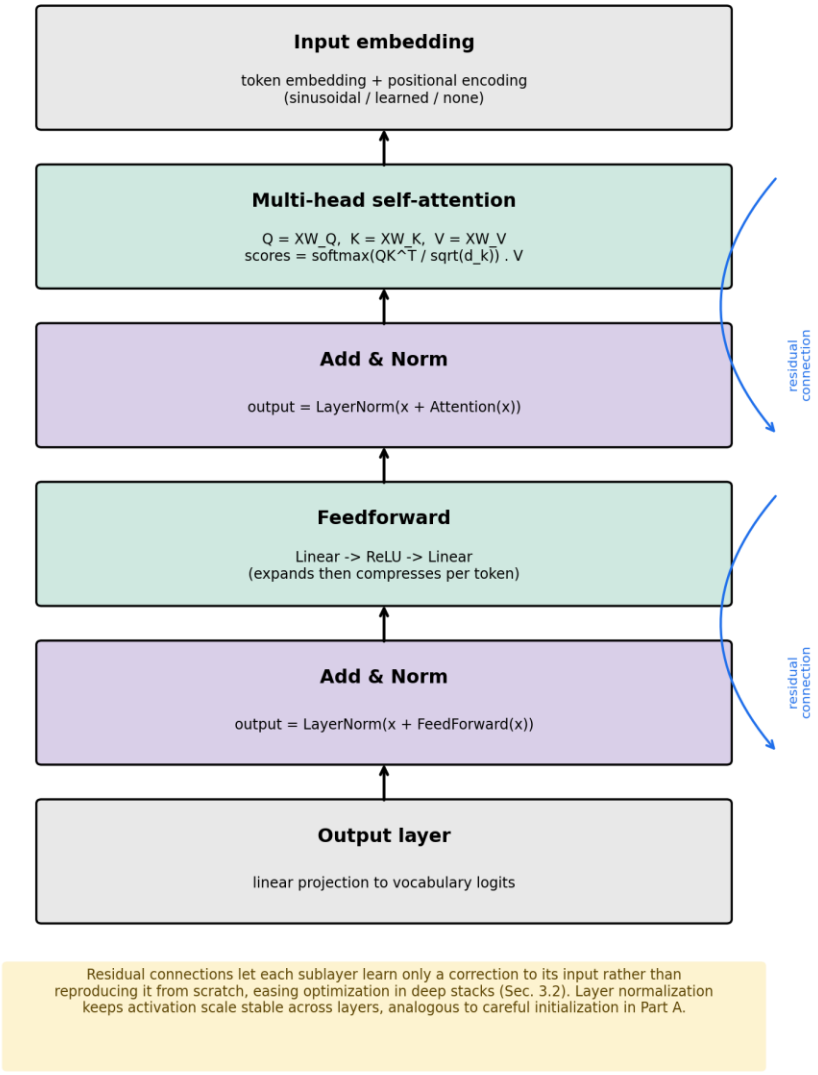

Figure 3. Transformer block architecture (repeated for each of the two stacked blocks).

3.3 Positional Encoding Schemes

Three schemes were compared. Sinusoidal encoding, the original scheme proposed by Vaswani et al. [1], assigns each position a fixed vector composed of sine and cosine functions at geometrically increasing wavelengths. This encoding is deterministic and requires no additional learned parameters. Learned encoding treats position identically to token identity, as an index into a trainable embedding table optimized jointly with the rest of the network. NoPE supplies no positional information at all; the model receives only token embeddings, and any sensitivity to order must arise indirectly, if at all.

3.4 Experimental Design

Tasks.

Two synthetic tasks were designed to differ in how strongly the correct output depends on token position. Reversal requires the model to output an input sequence of digits in reverse order. This task is inherently positional: the correct symbol at each output position is determined entirely by its position in the input, and shuffling the input changes every element of the correct output. Counting requires the model to output the number of times a specified target digit occurs in an input sequence. This task is largely position-independent, since shuffling the input sequence does not change the correct output. Counting was chosen specifically as a contrast to reversal, to test whether findings regarding positional encoding generalize across tasks with different structural dependence on position, or are specific to inherently positional tasks.

For both tasks, models were trained exclusively on short sequences and evaluated, without further training, on sequences substantially longer than any seen during training. This is the standard experimental design for studying length generalization [2], [3].

A methodological refinement.

Reversal (Section 3.5) was trained on a single fixed sequence length of six tokens. An initial implementation of the counting task used the same design, but a diagnostic check, evaluating the trained model on sequences shorter than, as well as longer than, the training length, revealed that accuracy peaked sharply at exactly the trained length and degraded in both directions. This indicated that the model had overfit to a single specific length, rather than learning a mechanism that was simply insensitive to length up to some point and then failed beyond it. A single-length training design conflates these two distinct phenomena. The counting experiment was consequently redesigned to train on a range of lengths (three to six tokens, sampled uniformly per batch) rather than a single fixed length, isolating genuine extrapolation beyond the trained range from overfitting to one specific length. This revision, and the reasoning behind it, is reported here in the interest of transparency, and is treated as a limitation of the reversal experiment in Section 5.

3.5 Experiment 1: Reversal

Table II and Figure 4 report exact-match accuracy on the reversal task across sequence lengths, for models trained exclusively on length-6 sequences.

Length	Sinusoidal	Learned PE	NoPE
6 (trained)	1.000	1.000	~0.35-0.40
8	0.213	0.254	not evaluated
10	0.166	0.141	not evaluated

12	0.247	0.172	not evaluated
15	0.108	0.137	not evaluated
20	0.128	0.130	not evaluated
25	0.153	0.119	not evaluated

Table II. Reversal task accuracy by sequence length.

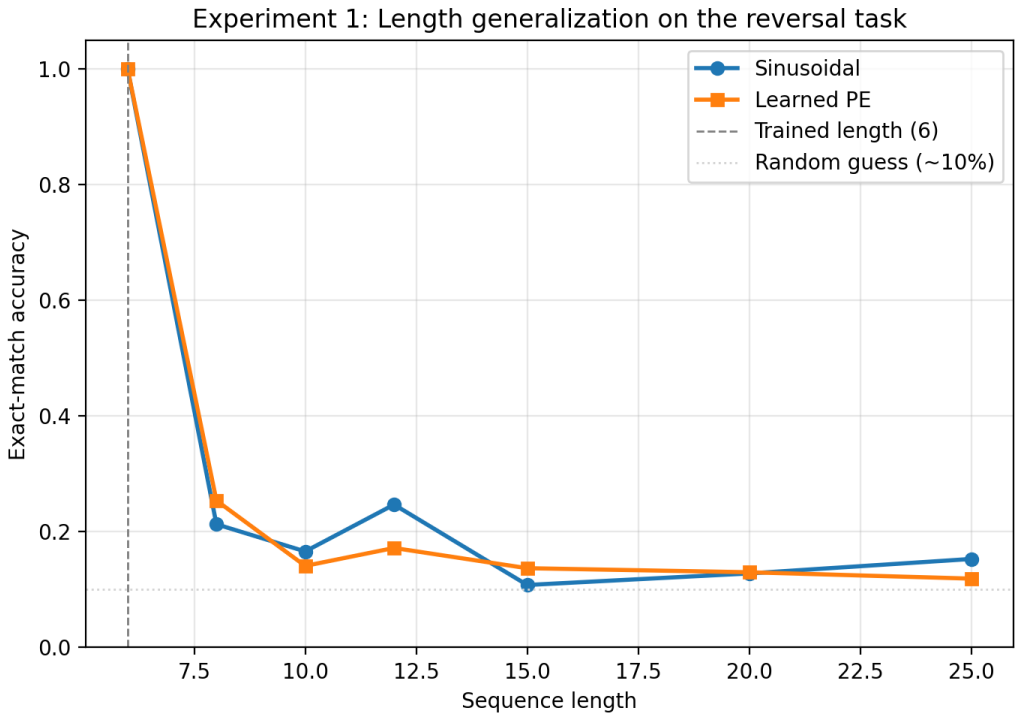

Figure 4. Length generalization on the reversal task.

NoPE failed to learn the task at all, plateauing at 35 to 40% training accuracy even at the trained length, well short of the near-perfect accuracy achieved by both other schemes. This is consistent with the task's structure. Because the correct output at each position depends entirely on that position's location in the input, a model with no mechanism for representing position cannot, in principle, solve it, regardless of training duration.

Sinusoidal and learned positional encoding performed near-identically. Both reached 100% accuracy at the trained length and both collapsed to near-chance performance (11 to 25%, against a 10% random baseline) immediately beyond it. The decline is abrupt rather than gradual: accuracy at length 8, only two tokens beyond training, is already close to the level observed at length 25. For this task, the encoding scheme's specific form appeared not to matter. What mattered was whether an explicit positional signal was present at all.

3.6 Experiment 2: Counting

Table III and Figure 5 report results on the counting task, trained on the length range 3 to 6 as described in Section 3.4.

Length	Sinusoidal	Learned PE	NoPE
3 (in range)	1.000	1.000	0.935
4 (in range)	1.000	1.000	0.955
5 (in range)	0.995	1.000	0.820
6 (in range)	0.970	0.995	0.600
8	0.690	0.735	0.215
10	0.170	0.510	0.080
12	0.070	0.295	0.095
15	0.030	0.125	0.020
20	0.025	0.060	0.040
25	0.005	0.025	0.010

Table III. Counting task accuracy by sequence length.

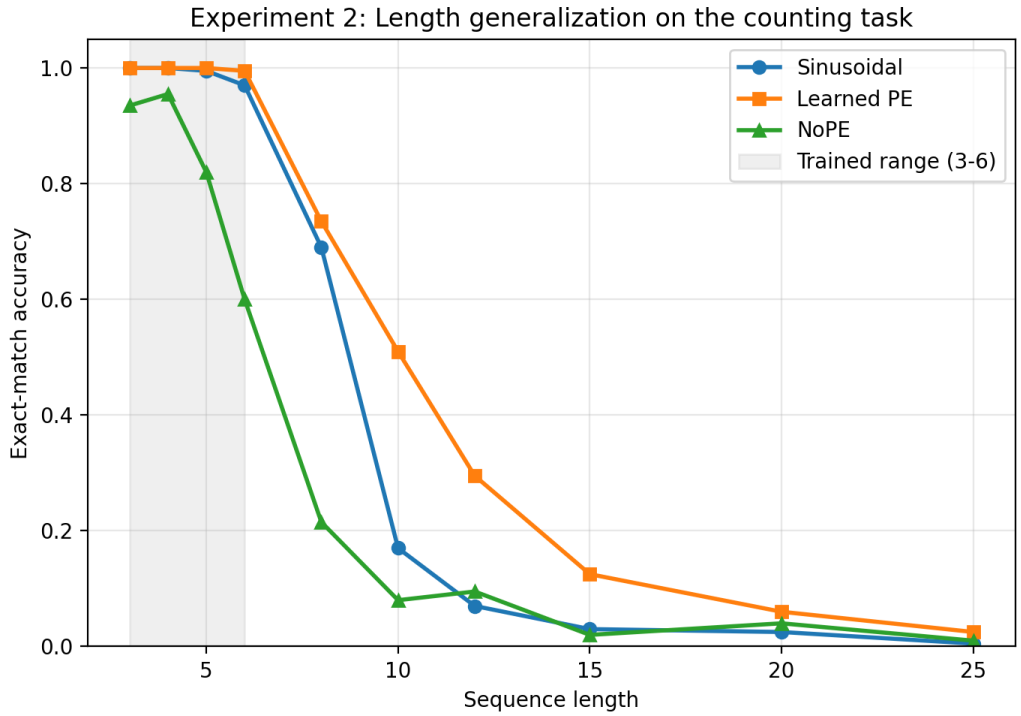

Figure 5. Length generalization on the counting task.

Within the trained range, all three schemes achieved high accuracy, though NoPE trailed the other two (60 to 95.5%, versus 97 to 100% for sinusoidal and learned PE) and exhibited visibly noisier training dynamics. Unlike in the reversal task, NoPE was able to learn a substantial majority of the

counting task. This is consistent with the hypothesis that positional information is necessary only to the extent that a task genuinely depends on position.

Beyond the trained range, sinusoidal and learned PE diverged clearly, in contrast to their near-identical behavior on reversal. Learned PE retained meaningfully higher accuracy at every extrapolated length tested (for example, 51.0% versus 17.0% at length 10, and 29.5% versus 7.0% at length 12), indicating that the relative performance of positional encoding schemes on length generalization is itself task-dependent rather than fixed. All three schemes ultimately degraded substantially by length 15 and beyond, indicating that none achieved robust generalization to sequences much longer than those seen during training.

3.7 Cross-Experiment Synthesis

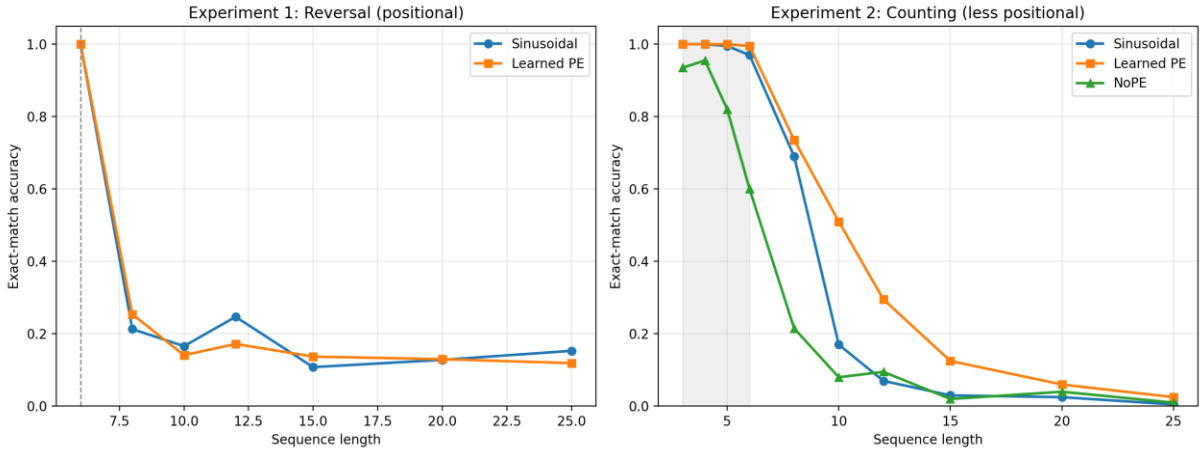

Figure 6. Combined comparison across both experiments.

Figure 6 presents both experiments side by side. Two conclusions follow from their joint consideration. First, whether positional encoding is necessary at all is task-dependent: essential for a strongly positional task such as reversal, merely beneficial for a weakly positional one such as counting. Second, the relative ranking of positional encoding schemes is also task-dependent. Sinusoidal and learned PE were indistinguishable on reversal but diverged substantially on counting. Neither conclusion is visible from either experiment in isolation, which motivates the paired-task design used here and suggests that single-task evaluations of positional encoding schemes, common in the literature this work engages with [2], [3], may understate how sensitive such comparisons are to task structure.

4. Related Work

The Transformer and positional encoding.

Vaswani et al. [1] introduced the Transformer architecture and the sinusoidal positional encoding scheme used as one baseline in this work. Because self-attention is permutation-invariant by construction, some mechanism for injecting order information is a structural requirement of the architecture, not an optional addition. Subsequent work has proposed numerous alternatives to the original sinusoidal scheme, including learned absolute embeddings, relative position representations, and rotary embeddings, motivated variously by improved performance, training stability, or generalization behavior.

Length generalization.

Press et al. [2] documented that Transformers trained with sinusoidal or rotary positional encodings degrade substantially when evaluated on sequences longer than those seen during training, and introduced Attention with Linear Biases (ALiBi) as a remedy. ALiBi replaces explicit positional embeddings with a fixed, distance-dependent penalty applied directly to attention scores, and was shown to allow models trained on short sequences to extrapolate effectively to substantially longer ones. This result established length generalization as a property that depends materially on encoding choice, rather than being a fixed limitation of the Transformer architecture as a whole.

A more recent and broader study by Kazemnejad et al. [3] complicates this picture. Evaluating five positional encoding schemes, including ALiBi, rotary embeddings, and learned absolute embeddings, on a suite of algorithmic and reasoning tasks, this work found that a decoder-only Transformer supplied with no positional encoding at all (NoPE) matched or outperformed every explicit scheme tested on length generalization, suggesting that decoder-only models can infer sufficient positional information implicitly, through the causal masking structure of autoregressive attention, without an explicit encoding. This finding stands in some tension with [2] and with the broader premise that an explicit, well-designed encoding scheme is necessary for extrapolation.

The present work engages with this tension directly, though from a different angle than either study. Rather than testing additional encoding schemes at larger scale, it holds the set of schemes relatively simple (sinusoidal, learned, and no encoding) and instead varies the task, finding that both the necessity of positional encoding and the relative ordering of schemes shift depending on how strongly a task depends on position. This is consistent with the possibility that part of the disagreement between [3] and [2] reflects differences in the tasks evaluated (reasoning and arithmetic tasks in [3] versus more directly positional extrapolation tasks in [2]) rather than a straightforward disagreement about which encoding scheme is superior in general.

Alternative approaches to length generalization.

Beyond ALiBi, Ruoss et al. [7] proposed randomizing relative positional offsets during training, to prevent a model from overfitting to the specific range of positions observed. This approach

shares a common premise with the present work: that a model's positional representation, and its relationship to the exact lengths seen during training, is a primary lever for controlling extrapolation behavior, independent of the base architecture.

5. Discussion

5.1 Connecting Part A and Part B

Although Part A and Part B examine different architectures and different empirical questions, they share a common methodological thread. In both cases, an initial implementation produced a result that could easily have been misread. The dead-ReLU collapse in Part A (Section 2.2) produced a fixed, uninformative accuracy value that could plausibly have been mistaken for a bug in the training loop rather than a genuine, explainable property of the ReLU and initialization interaction. Similarly, the initial single-length version of the counting experiment in Part B (Section 3.4) produced a result that, without the additional diagnostic step of testing shorter as well as longer sequences, could have been reported as straightforward evidence of length generalization failure, when it in fact reflected overfitting to a single training length. In both instances, a deliberate step of verification, checking whether a surprising result was internally consistent and mechanistically explicable before treating it as a finding, changed the conclusion drawn. This suggests a general methodological point applicable beyond either specific experiment: an unexpected result in a from-scratch implementation warrants the same scrutiny whether it appears convenient or inconvenient for the hypothesis being tested.

5.2 Limitations

Several limitations of the experiments in Part B should be noted. First, results are reported from a single trained model per condition rather than averaged over multiple random seeds; the magnitude of run-to-run variance is therefore not directly quantified, and some of the smaller differences reported (for instance, between sinusoidal and learned PE at intermediate lengths in Experiment 1) should be interpreted cautiously. Second, the reversal experiment used a single fixed training length, while the counting experiment used a range of training lengths, following the revision described in Section 3.4; this inconsistency was not retrospectively corrected in the reversal experiment, and a fully consistent range-based design across both experiments would strengthen a direct comparison between them. Third, only two positional encoding schemes with explicit positional information were tested, alongside a no-encoding control. Relative schemes such as ALiBi or rotary embeddings were not implemented, and their inclusion would allow a more direct comparison with the specific schemes evaluated in [2] and [3]. Fourth, the architecture used throughout is intentionally small (two layers, model dimension 32), consistent with the goal of a from-scratch, fully interpretable implementation, but leaves open the extent to which the task-

dependence reported here persists at the scale of models typically studied in the length generalization literature.

5.3 Implications and Future Work

The central empirical claim of this work, that both the necessity of positional encoding and the relative ranking of encoding schemes depend on task structure, suggests that single-task benchmarking of positional encoding schemes may be an incomplete basis for general claims about length generalization. A natural extension would test a broader range of tasks, spanning a spectrum of positional dependence rather than the two endpoints considered here, together with the relative encoding schemes noted above, and with multiple random seeds per condition to establish the statistical reliability of the differences observed.

6. Conclusion

This work implemented a fully connected neural network and a Transformer encoder from first principles, and used each as the basis for a concrete investigation. The neural network implementation surfaced and resolved a specific, mechanistically understood failure mode arising from the interaction of activation function, weight initialization, and learning rate, and supported a small ablation study identifying mini-batch gradient descent as a substantial source of improvement over full-batch training. The Transformer implementation was used to study length generalization across two tasks differing in their dependence on token position, finding that neither the necessity of positional encoding nor the relative performance of different encoding schemes is fixed across tasks. Taken together, these results support the view that careful, first-principles implementation is not only a pedagogical exercise, but a basis from which genuine, if modest, empirical contributions can be made.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[2] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in Proc. Int. Conf. on Learning Representations (ICLR), 2022.

[3] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, and S. Reddy, "The impact of positional encoding on length generalization in transformers," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in Proc. IEEE Int. Conf. on Computer Vision (ICCV), 2015.

[5] S. Sabour, N. Frosst, and G. E. Hinton, "Dynamic routing between capsules," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2016.

[7] A. Ruoss, G. Deletang, T. Genewein, J. Grau-Moya, R. Csordas, M. Bennani, S. Legg, and J. Veness, "Randomized positional encodings boost length generalization of transformers," in Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023.